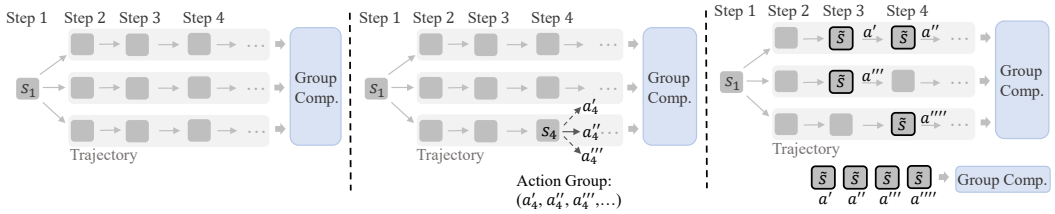

Figure 1: Comparison of long-horizon LLM agent training. **Left**: Vanilla GRPO rolls out a group of full trajectories and computes episode-level advantages. **Middle**: Constructing step-level groups via additional per-state rollouts (e.g., $a_4', a_4'', a_4''', \ldots$) enables fine-grained feedback but incurs prohibitive computational cost. **Right**: GiGPO efficiently achieves fine-grained credit assignment by aggregating distinct actions (a', a'', a''', a'''') taken from the same environment state $\tilde{s}$ across the trajectories.

the substantial overhead of extra LLM forward passes and the difficulty of evaluating rewards for hypothetical actions never actually executed.

To overcome these challenges, we propose our *Group-in-Group Policy Optimization (GiGPO)* in this section. Similar to prior works [48, 50], GiGPO begins by sampling groups of trajectories under identical tasks and initial environment states. It then introduces a two-level grouping structure: preserving episode-level grouping for holistic performance comparison, while dynamically constructing an additional set of step-level groups by retroactively aggregating actions encountering the same environment states. This "group-in-group" construction yields two complementary advantages: (1) *Episode relative advantages* capture the holistic effectiveness of each trajectory, providing a stable, global training signal. (2) *Step relative advantages* zoom in on which actions outperform their peers within the same state, endowing the gradient with fine-grained credit.

Figure 2 presents an overview of the GiGPO training pipeline. In the remainder of this section, we will detail the computation of episode-level relative advantages, elaborate on the anchor state grouping mechanism, describe the derivation of step-level relative advantages, and finally present the overall GiGPO objective.

4.1 Episode Relative Advantages

We first introduce the episode-level relative advantages, which represent the coarse-grained component of GiGPO and mirror the naive application of GRPO at the trajectory level. We roll out the agent's policy $\pi_{\theta_{\text{old}}}$ in the environment to collect N complete trajectories under a fixed task x and identical initial states. Formally, this process yields a group of trajectories $\{\boldsymbol{\tau}_i\}_{i=1}^N$, where each trajectory is denoted as $\boldsymbol{\tau}_i = \{(\boldsymbol{s}_1^{(i)}, \boldsymbol{a}_1^{(i)}, r_1^{(i)}), \ldots, (\boldsymbol{s}_T^{(i)}, \boldsymbol{a}_T^{(i)}, r_T^{(i)})\}$ and the initial states satisfy $\boldsymbol{s}_1^{(1)} = \boldsymbol{s}_1^{(2)} = , \ldots, = \boldsymbol{s}_1^{(N)}$. For each trajectory, we utilize the total return $R(\boldsymbol{\tau}_i) = \sum_t r_t^{(i)}$ as a holistic measure of how effectively the agent completes the task. In settings where only a binary reward is given at the end of the episode, this simplifies to $R(\boldsymbol{\tau}_i) = 1$ for success and $R(\boldsymbol{\tau}_i) = 0$ for failure. Then, we organize the trajectories and their corresponding returns into an episode-level group:

$$G^E = \Big\{ \big(\boldsymbol{\tau}_1, R(\boldsymbol{\tau}_1)\big), \big(\boldsymbol{\tau}_2, R(\boldsymbol{\tau}_2)\big), \ldots, \big(\boldsymbol{\tau}_N, R(\boldsymbol{\tau}_N)\big) \Big\}. \tag{2}$$

To evaluate the global relative quality of each trajectory within the group, we compute an episode relative advantage $A^E(\boldsymbol{\tau}_i)$ for each $\boldsymbol{\tau}_i$ by normalizing the total return with the group's mean and a normalization factor:

$$A^E(\boldsymbol{\tau}_i) = \frac{R(\boldsymbol{\tau}_i) - \text{mean}\big(\{R(\boldsymbol{\tau}_j)\}_{j=1}^N\big)}{F_{\text{norm}}\big(\{R(\boldsymbol{\tau}_j)\}_{j=1}^N\big)}. \tag{3}$$

In GRPO [17], the default normalization factor is defined as the standard deviation, i.e., $F_{\text{norm}} = \text{std}$. However, this may introduce a difficulty bias [18], where trajectories from low-variance groups (e.g., very easy or hard tasks) receive disproportionately large gradients. In the context of the LLM agent, where tasks often involve very long horizons, this effect tends to emerge frequently, potentially affecting the stability of updates. As an alternative, we also consider a fixed normalization factor $F_{\text{norm}} = 1$, which yields an unbiased Leave-One-Out estimator [15] (see Appendix B for details). This simple adjustment helps stabilize training in some challenging agent scenarios.

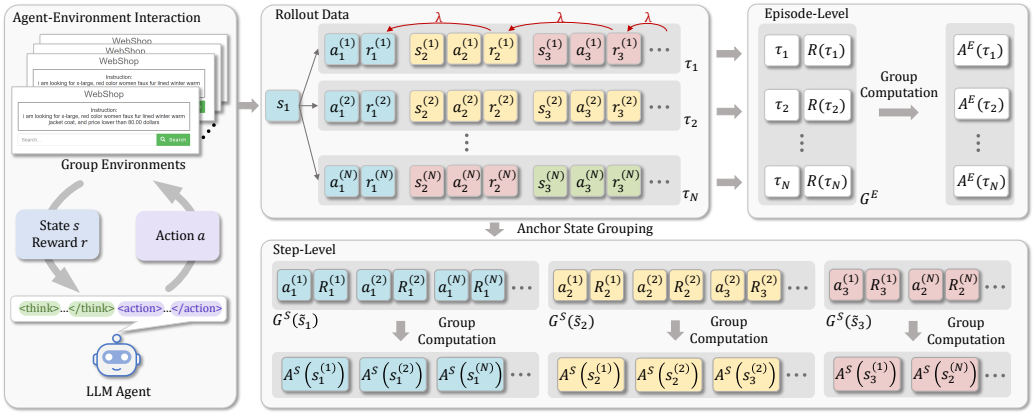

Figure 2: Overview of GiGPO. The agent interacts with a group of environments initialized with identical states to generate a set of trajectories $\{\boldsymbol{\tau}_i\}_{i=1}^{N}$. States with the same color represent the same environment state. GiGPO performs two-dimensional group computations (episode-level A^E and step-level A^S) to produce hierarchical relative advantages that guide fine-grained policy optimization.

Overall, the episode relative advantage captures whether the agent successfully completes the assignment across the entire decision horizon T. Similar to the vanilla GRPO for multi-step optimization shown in Figure 1 (left), it primarily focuses on macro credit assignment, encouraging the policy to develop coherent, trajectory-wide behaviors that maximize overall task performance.

4.2 Step Relative Advantages

While the episode relative advantage offers a macro, trajectory-wide signal, it cannot distinguish between the contributions of individual actions within the trajectory. To obtain this fine-grained feedback, we need to form step-level groups: for the same state, we gather the different actions and compare their outcomes, thereby learning which choices are relatively better or worse. A naive way to do so would be to roll out fresh actions from every state (Figure 1, middle), but that is prohibitively expensive. Instead, we introduce anchor state grouping below, avoiding extra LLM overhead.

Anchor state grouping. As all trajectories $\{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_N\}$ arise from the same task x and identical initial conditions, many environment states naturally recur across episodes and even across time steps within a single trajectory. We leverage this redundancy by identifying and grouping identical states across trajectories, thereby dynamically constructing step-level groups. Specifically, let $\mathcal{U} = \{\tilde{\boldsymbol{s}}_1, \tilde{\boldsymbol{s}}_2, \ldots, \tilde{\boldsymbol{s}}_U\}$ denote the set of all distinct environment states appearing in the trajectory group $\{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_N\}$. We treat each such unique state $\tilde{\boldsymbol{s}} \in \mathcal{U}$ as an implicit anchor and use it to gather all matching occurrences of that state, and therefore call $\tilde{\boldsymbol{s}}$ as "*anchor state*". Based on this, we can construct $|\mathcal{U}|$ step-level groups (one for each unique anchor state $\tilde{\boldsymbol{s}}$), which is defined as follows:

$$G^S(\tilde{\boldsymbol{s}}) = \left\{ \left(\boldsymbol{a}_t^{(i)}, r_t^{(i)}\right) \;\middle|\; \boldsymbol{s}_t^{(i)} = \tilde{\boldsymbol{s}},\ 1 \le i \le N,\ 1 \le t \le T \right\}. \tag{4}$$

Unlike per-state rollout, this procedure incurs no extra rollouts: it is entirely offline and requires only lightweight key-based grouping using hashmaps. Each group $G^S(\tilde{\boldsymbol{s}})$ contains multiple instances of the same environment state paired with potentially different actions. Hence, this structure effectively builds the step-level group, forming the basis for subsequent step-level advantage estimation.

Relative advantage computation. Although each tuple $\left(\boldsymbol{a}_t^{(i)}, r_t^{(i)}\right)$ contains an immediate reward $r_t^{(i)}$, it may be sparse, especially in long-horizon tasks. To better capture long-term impact, we associate a *discounted return* with each step. Let $\gamma \in (0, 1]$ be the standard RL discount factor. For each element in $G^S(\tilde{\boldsymbol{s}})$, we compute its discounted return $R_t^{(i)}$ by

$$R_t^{(i)} = \sum_{k=t}^{T} \gamma^{k-t} r_k^{(i)}. \tag{5}$$

This quantity captures the future impact of action $\boldsymbol{a}_t^{(i)}$ on subsequent rewards, rather than relying solely on the immediate reward $r_t^{(i)}$. Accordingly, the step-level group for each $\tilde{\boldsymbol{s}} \in \mathcal{U}$ becomes:

$$G^S(\tilde{\boldsymbol{s}}) = \left\{ \left(\boldsymbol{a}_t^{(i)}, R_t^{(i)}\right) \;\middle|\; \boldsymbol{s}_t^{(i)} = \tilde{\boldsymbol{s}},\ 1 \le i \le N,\ 1 \le t \le T \right\}. \tag{6}$$

Once these step-level groups are formed, we compute the *step relative advantage* for each $\tilde{\boldsymbol{s}} \sim \mathcal{U}$ and each action $\boldsymbol{a}_t^{(i)}$ in $G^S(\tilde{\boldsymbol{s}})$:

$$A^S(\boldsymbol{a}_t^{(i)}) = \frac{R_t^{(i)} - \text{mean}\left(\left\{R_t^{(j)} \mid (\boldsymbol{a}_t^{(j)}, R_t^{(j)}) \in G^S(\tilde{\boldsymbol{s}})\right\}\right)}{F_{\text{norm}}\left(\left\{R_t^{(j)} \mid (\boldsymbol{a}_t^{(j)}, R_t^{(j)}) \in G^S(\tilde{\boldsymbol{s}})\right\}\right)}. \tag{7}$$

A^S provides micro credit assignment and fine-grained feedback on the relative quality of individual actions taken from the same state. In contrast to the coarse, trajectory-wide signal of A^E, it offers step-level guidance that is essential for refining decisions in long-horizon agent tasks.

How does step-level group work? We present an intuitive illustration in Figure 3 to show the utility of the step relative advantages. We consider two example trajectories from the set $\{\boldsymbol{\tau}_i\}_{i=1}^N$. In τ_1, the agent first selects the *2nd Item* (incorrect), then returns to the previous page and selects the *1st Item* (correct), successfully completing the task. Due to temporal discounting (Equation (5)), the earlier action (*2nd Item*) receives a lower discounted return than the later correct one (*1st Item*). In τ_2, the agent clicks the *Next Page*, ultimately failing to find the target and receiving no reward. By aggregating these actions into the same step-level group based on their shared anchor state, GiGPO computes their relative advantages and yields a clear preference ordering: $A^S(\textit{1st Item}) > A^S(\textit{2nd Item}) > A^S(\textit{Next Page})$. This ranking successfully captures fine-grained distinctions in long-term utility that are missed by prior group-based RL methods [16, 17, 19]. While this example illustrates GiGPO's effectiveness in sparse-reward environments, its advantages extend naturally to dense-reward scenarios, where per-step rewards can be fully leveraged to assess the relative quality of individual actions within shared states.

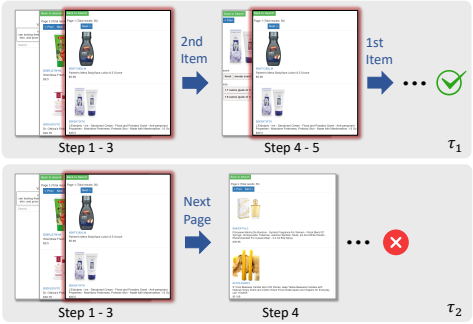

Figure 3: Illustration of step-level grouping in WebShop. Both τ_1 and τ_2 encounter the same environment state multiple times: a search results page (highlighted by the red border). **Top**: τ_1 eventually succeeds. **Bottom**: τ_2 leads to failure.

4.3 Group-in-Group Policy Optimization

We finally combine the two levels of advantage signals into a single *group-in-group advantage* to assign credit at both global (episode) and local (step) scales:

$$A(\boldsymbol{a}_t^{(i)}) = A^E(\boldsymbol{\tau}_i) + \omega \cdot A^S(\boldsymbol{a}_t^{(i)}), \tag{8}$$

where $\omega \in \mathbb{R}_{\geq 0}$ is a weighting coefficient that balances episode relative advantage and step relative advantage. $A^E(\boldsymbol{\tau}_i)$ captures how good an episode is compared to others in the group, while $A^S(\boldsymbol{a}_t^{(i)})$ refines step-level performance within shared environment state conditions. Jointly, they provide hierarchical supervision for the policy optimization of LLM agents. Then the clipped policy optimization objective of GiGPO is:

$$\begin{aligned}\mathcal{J}_{\text{GiGPO}}(\theta) = \mathbb{E}_{\substack{x \sim p(X) \\ \{\boldsymbol{\tau}_i\}_{i=1}^N \sim \pi_{\theta_{\text{old}}}}} \left[\frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T} \min\Big(\rho_\theta(\boldsymbol{a}_t^{(i)})A(\boldsymbol{a}_t^{(i)}),\, \text{clip}\big(\rho_\theta(\boldsymbol{a}_t^{(i)}), 1 \pm \epsilon\big)A(\boldsymbol{a}_t^{(i)})\Big)\right] \\ - \beta \mathbb{D}_{\text{KL}}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big).\end{aligned} \tag{9}$$

where $\rho_\theta(\boldsymbol{a}_t^{(i)}) = \frac{\pi_\theta(\boldsymbol{a}_t^{(i)} \mid \boldsymbol{s}_t^{(i)}, x)}{\pi_{\theta_{\text{old}}}(\boldsymbol{a}_t^{(i)} \mid \boldsymbol{s}_t^{(i)}, x)}$ is the importance sampling ratio, β controls the strength of the KL penalty encouraging proximity to a reference policy π_{ref}. We present the pseudo code in Appendix C.

5 Experiment

In this section, we present empirical evaluations of GiGPO across a variety of LLM agent tasks. Specifically, we aim to demonstrate: **(1)** the strong ability of GiGPO in training LLM agents; **(2)** the ablation study of GiGPO; **(3)** the dynamic trend of step-level group $G^S(\tilde{\boldsymbol{s}})$ over the course of training; **(4)** the computational budget of GiGPO.